

Module 5: Data Processing using NumPy and Pandas

Premanand S

Assistant Professor Junior
School of Electronics Engineering
Vellore Institute of Technology, Chennai Campus

premanand.s@vit.ac.in

October 13, 2025

Introduction

- Till now, we learned Python using simple values like numbers, strings, and lists.
- We wrote programs to do calculations, make decisions, and repeat tasks using loops.
- But everything we handled was small and manual — like working with a few values at a time.
- In the real world, a Data Scientist works with large amounts of data — in rows and columns, just like an Excel sheet.

Real-World Data Example

- Think of a dataset like this:

Name	Age	Marks	City
Asha	18	85	Chennai
Ramesh	19	78	Delhi
John	17	92	Mumbai

- To work with this kind of data, we need special tools to read, clean, and analyze it easily.

Why We Need NumPy and Pandas

- Data can be large, complex, and come from real-world sources like files or sensors.
- We need tools that can:
 - Handle lots of numbers quickly
 - Perform math operations easily
 - Manage tabular data like Excel
 - Read and analyze data from files
- This is where **NumPy** and **Pandas** come into play.

What is NumPy?

- **NumPy** stands for *Numerical Python*.
- It helps to work with large arrays of numbers.
- Acts like a supercharged calculator that can handle many numbers at once.
- It is the backbone for mathematical and scientific operations in Python.
- Example: Adding two lists of numbers in a single step.

Example: NumPy Array Addition

```
import numpy as np

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

result = a + b
print(result)    # Output: [11 22 33]
```

What is Pandas?

- **Pandas** stands for *Python Data Analysis Library*.
- It helps to work with data in rows and columns (like Excel sheets).
- Provides special structures like:
 - **Series** – one-dimensional data
 - **DataFrame** – two-dimensional data (rows and columns)
- With Pandas, we can read, clean, filter, sort, group, and analyze data easily.

Example: Creating a DataFrame in Pandas

```
import pandas as pd

data = {
    "Name": ["Asha", "Ramesh", "John"],
    "Age": [18, 19, 17],
    "Marks": [85, 78, 92]
}

df = pd.DataFrame(data)
print(df)
```


PIP

Introduction to pip

- Before using external libraries like NumPy or Pandas, we need a way to install them.
- **pip** is the tool used to install and manage Python packages.
- The word **pip** stands for *"Pip Installs Packages"*.
- It allows us to download and install thousands of ready-made Python libraries from the internet.

Why Do We Need pip?

- Python itself provides only the basic features.
- To do special tasks (like data analysis, web development, or machine learning), we need extra tools called **libraries**.
- These libraries are not built into Python — they must be installed separately.
- The **pip** command helps us download and install these libraries easily from the Python Package Index (**PyPI**).

How pip Works

- Think of **pip** as a delivery service for Python packages
- When we type a command like:

```
pip install numpy
```

- pip connects to the online Python Package Index (PyPI)
- Downloads the library files
- Installs them into your system's Python environment

Checking pip Installation

- Usually, **pip** comes pre-installed with Python.
- You can check if pip is installed by typing this in Command Prompt or Terminal:

```
pip --version
```

- Example output:
 pip 24.0 from C:\Python311\Lib\site-packages\pip
- This shows that pip is installed and ready to use.

Upgrading pip (Optional)

- You can upgrade pip itself to the latest version by typing:

```
pip install --upgrade pip
```

- It is good practice to keep pip updated to avoid compatibility issues.

NumPy

Installing NumPy

- Open your Command Prompt (Windows) or Terminal (Mac/Linux).
- Type the following command:

```
pip install numpy
```

- This command downloads and installs the latest version of NumPy from Python Package Index (PyPI).
- Once installed, you can start using NumPy in your Python programs.

Checking NumPy Version

- After installation, you can check the installed version of NumPy.
- Run this command in Python:

```
import numpy as np  
print(np.__version__)
```

- This will display the current version of NumPy installed on your system.

Upgrading NumPy to the Latest Version

- To upgrade NumPy to the latest available version, use:

```
pip install --upgrade numpy
```

- The `--upgrade` flag tells pip to install the newest version available.
- Example output may show the new version being downloaded and installed.

Downgrading (Installing a Specific Version)

- Sometimes, you may need an older version for compatibility with other libraries.
- You can install a specific version using:

```
pip install numpy==1.21.0
```

- Here, 1.21.0 is the version number.
- Replace it with any version you wish to install.

Checking All Installed Packages

- You can list all installed Python packages and their versions using:

```
pip list
```

- This helps you verify if NumPy is installed correctly.
- You will see an entry like: `numpy 1.26.0`

Exploring Functions and Modules in NumPy

- Once NumPy is installed, you can explore its built-in functions.
- Run the following commands in Python:

```
import numpy as np
```

```
# List all functions and classes inside NumPy  
print(dir(np))
```

- The `dir()` function lists all available functions, methods, and classes in the NumPy module.
- This helps you explore what NumPy can do.

Getting Help for a Function

- You can get help on any specific NumPy function using the `help()` command.
- Example:

```
import numpy as np  
help(np.array)
```

- This displays the purpose, parameters, and examples for the `array()` function.
- Very useful when learning new functions.

Numpy - Array

What is an Array?

- An **array** is a collection of values stored in a single variable.
- It helps us store and manage multiple items of the same type (like numbers) together.
- Example: Instead of creating many variables a_1 , a_2 , a_3 , ..., we can store all values in one array.
- Arrays make mathematical operations easier and faster.

Example: Without Array

```
# Using normal variables  
a1 = 10  
a2 = 20  
a3 = 30  
  
total = a1 + a2 + a3  
print(total)
```

- Here, we manually add each variable.
- This becomes difficult when we have hundreds of values.

Example: With List (Still Not Efficient)

```
numbers = [10, 20, 30, 40, 50]
total = sum(numbers)
print(total)
```

- Lists can store multiple items easily.
- But Python lists are slow for large-scale mathematical operations.
- We need a faster and more memory-efficient structure.

Why NumPy Arrays?

- **NumPy Arrays** are faster, more compact, and more powerful than Python lists.
- They allow:
 - Fast mathematical operations on entire arrays
 - Element-wise computations without loops
 - Handling large numerical datasets efficiently
- NumPy arrays form the base for almost all data science and machine learning operations.

Example: NumPy Array in Action

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])

print("Addition:", a + b)
print("Multiplication:", a * b)
```

- NumPy can perform array operations directly — no need for loops!
- Each operation happens element-by-element, quickly and efficiently.

Visualizing List vs NumPy Array

- Python **lists** are flexible but slow for large numerical data.
- **NumPy arrays** are faster and use less memory.
- Let's visualize the difference using a simple program.

Execution Time Comparison

```
import numpy as np
import time

# Create a list and a numpy array with 1 million numbers
size = 1_000_000
list1 = range(size)
array1 = np.arange(size)

# Time taken by list
start = time.time()
list_result = [x * 2 for x in list1]
print("List Time:", time.time() - start)
```

Execution Time Comparison

```
# Time taken by numpy array
start = time.time()
array_result = array1 * 2
print("NumPy Time:", time.time() - start)
```

- Output shows NumPy is much faster for numerical operations.

Memory Usage Comparison

```
import numpy as np
import sys

# Create a list and a numpy array
list1 = [1, 2, 3, 4, 5]
array1 = np.array([1, 2, 3, 4, 5])

print("List size:", sys.getsizeof(list1))
print("NumPy array size:", array1.nbytes)
```

- Lists take more memory because each element is a separate Python object.
- NumPy arrays store data in continuous memory blocks — more compact and efficient.

Observation

- For small data, lists and NumPy arrays look similar.
- But for large data:
 - NumPy performs operations much faster
 - NumPy consumes less memory
- Hence, NumPy is preferred in Data Science for handling large datasets efficiently.